Why We Stopped the Attempt 4 Prefix-Code Direction

What the closest papers already cover

Next-meeting draft — not yet presented

2026-07-22

1. The Decision in One Sentence

We wanted one compact database code to support two jobs:

- read only the first few bits for a cheap first pass; and
- read the complete code for a more accurate second pass.

The literature already supplies the factorized code, progressive bit meaning, learned label assignment, and two-pass search pattern.

Decision

Stop at the literature gate: `NO_GO_DIRECT_COMPOSITION`. No implementation or data was needed.

2. What the Proposed Code Was Supposed to Do

Factorized code Split a vector into small groups and quantize each group separately.

Prefix The first few bits of one stored code.

Coarse pass Use less information to reject unlikely candidates.

Fine pass Use the complete code for a better distance estimate.

$$\texttt{one fixed stored word} \rightarrow \begin{array}{c|c} \text{first } p \text{ bits} & \text{all } B \text{ bits} \\ \text{cheap score} & \text{accurate lookup} \end{array}$$

The hoped-for contribution was to design both meanings together. Terms used below: VQ = vector quantization; PQ = product quantization; OPQ = optimized PQ; ANN = approximate nearest-neighbor search; ADC = query-to-code lookup; k-means = standard centroid training.

3. The Review Question Was Narrow

We did not ask only whether a new optimizer could lower reconstruction error. We asked whether the two-stage code could be obtained by combining known factorized quantizers with known progressive or learned index assignments. A surviving direction needed at least one of:

1. a new constraint that prevents the known pieces from composing;
2. a new lemma or algorithm needed by that constraint; or
3. a measurable state or training advantage unavailable to block vector quantization.

The primary papers did not leave such a gap.

4. Papers That Already Cover the Fine Code

Muresan–Effros (2002): optimal scalar quantization by histogram segmentation; learned scalar values and an integer number of levels. **Brandt (2010)**: principal-component rotation, learned one-dimensional quantizers, integer bit allocation, packed words, lookup tables, and approximate nearest-neighbor scan. **Finite Scalar Quantization (2024)**: a product codebook made from small scalar level sets, including non-power-of-two factor sizes.

Already covered

Learn scalar values + choose level counts + form a product code + pack a word.

5. BAPQ: Unequal Bits Are Not Progressive Bits

Adaptive Bit Allocation Product Quantization (2016) calls its method BAPQ:

1. rotate with principal component analysis and split into subspaces;
2. give the next bit to the subspace that most reduces full reconstruction error;
3. fit that subspace codebook with standard centroid training (k-means); and
4. concatenate ordinary subspace codeword indices.

Five bits and three bits mean codebooks with 32 and 8 entries. The first three bits of the five-bit label are not a trained coarse code.

Boundary

BAPQ covers adaptive bit allocation, not a nested prefix or prefix/full joint training.

6. Riskin: Add Progressive Meaning Afterward

Riskin et al. (1994) begin with an existing full-search vector quantizer. Its finest encoding regions stay fixed.

$$\text{fixed fine regions} \rightarrow \text{merge regions} \rightarrow \text{fit an intermediate centroid}$$

The paper explicitly says it optimizes the decoder for a fixed encoder, not the encoder for the decoder. Intermediate decoding needs no new search: use the centroid of the merged region that contains the selected fine region.

Why it matters

Progressive prefixes are a post-processing step that can be placed over an arbitrary fixed fine codebook.

7. Derived Codebooks: The Closest ANN System

Derived Codebooks (2019) already applies the coarse/fine idea to product quantization for approximate nearest-neighbor search.

1. train a fine 16-bit product codebook;
2. derive an 8-bit grouping of its fine centroids;
3. reorder labels so selected bits identify the coarse group;
4. scan coarsely, then refine selected candidates accurately.

Its selector bits are not printed first, but a fixed bit permutation can move them to the front. That gives the same physical interface as a prefix.

8. Polysemous Codes: One Label, Two Meanings

Polysemous Codes (2016) first trains an ordinary product quantizer, then learns a one-to-one mapping from centroid indices to binary labels.

- Cheap meaning: compare bit strings with Hamming distance.
- Accurate meaning: use the original product centroids for distance lookup.

Its loss can preserve centroid distance or ranking. It uses all label bits rather than a nested leading prefix, but establishes the main mechanism: one stored label can support both cheap filtering and accurate scoring.

Training order

Product codebook first, binary mapping second.

9. Hierarchical Coding Was Also Checked

Chou–Lookabaugh–Gray (1989) jointly design and prune tree-structured vector quantizers under distortion and rate objectives. This prevents the broad claim that "joint hierarchical quantizer design is new." However, the paper studies variable-rate source coding, not one fixed random-access database word with a prescribed ANN lookup path.

Precise conclusion

No one paper contains our whole candidate. The missing fixed-word ANN construction is nevertheless obtained by composing the other papers, without a new coupling constraint.

10. Coverage Map

Paper family	Fine product code	Learned/progressive label	Cheap + accurate search	Joint prefix/full
Muresan, Brandt, FSQ	Yes	No	Brandt: accurate	No
OPQ, BAPQ, Quicker ADC	Yes	No	accurate PQ	No
Riskin	fixed fine VQ	Yes	progressive decode	No
Derived Codebooks	Yes	Yes	scan + refine	No
Polysemous Codes	Yes	Yes	filter + score	No
Chou et al.	hierarchical VQ	Yes	not fixed-word ANN	source coding

No row alone has every property. The decisive question is whether joining the rows requires a genuinely new mechanism. We found none.

10A. Additional Boundary Papers

- **OPQ (2013)** learns a rotation and product codebooks, but no progressive label meaning.
- **Quicker ADC (2021)** exposes packing and lookup costs for 4-, 5-, and 6-bit subcodes; sizes remain 2^b.
- **Q-Palette (2025)** covers fractional-rate quantizers and mixed allocation, but not this prefix consumer.
- **FibQuant (2026)** strengthens the block-vector baseline; it supplies no prefix mechanism.

These close coding and systems side routes. Riskin, Derived Codebooks, and Polysemous are the decisive prefix and two-meaning papers.

11. Why the Combination Is Direct

Known fine-code step

FSQ, Brandt, or BAPQ trains factorized representatives and chooses level counts.

Known two-meaning step

Riskin, Derived, or Polysemous groups or relabels trained fine codewords for a cheap pass.

Replacing ordinary product-quantization centroids with arbitrary-level scalar products changes reconstruction quality, model size, and training cost. It does not create a new prefix constraint, theorem, or search algorithm.

Meaning of the verdict

The proposal is a direct composition of known mechanisms.

12. What the Decision Does—and Does Not—Say

It does say

- this successor lacks mechanism-level novelty;
- arbitrary cardinality is unnecessary for the prefix mechanism;
- S1 implementation and data are not justified.

It does not say

- all progressive codes are useless;
- factorized codes never beat block codes;
- the proposal failed on ANN benchmarks.

Claim ceiling

This was an early literature stop, not an experimental failure.

13. What Would Be Needed to Reopen It

A reopening needs a different scientific claim, not another implementation of the same composition:

1. name a property unavailable to Riskin, Derived, Polysemous, and hierarchical VQ;
2. show the new constraint, lemma, or algorithm required by that property;
3. compare with independently optimized product and block vector quantization;
4. freeze construction, state, and query-cost gates before data.

Until such a gap is stated, keep these works as baselines and select a different problem.

Current authority

S0: `NO_GO_DIRECT_COMPOSITION`. S1: `NOT_AUTHORIZED`.

References and Decision Record I

- Muresan and Effros. “Quantization as Histogram Segmentation.” DCC 2002.
- Brandt. “Transform Coding for Fast Approximate Nearest Neighbor Search in High Dimensions.” CVPR 2010.
- Mentzer et al. “Finite Scalar Quantization.” ICLR 2024. https://arxiv.org/abs/2309.15505
- Ge et al. “Optimized Product Quantization.” CVPR 2013.
- Guo et al. “Adaptive Bit Allocation Product Quantization.” Neurocomputing, 2016. https://doi.org/10.1016/j.neucom.2015.07.062
- Riskin et al. “Index Assignment for Progressive Transmission of Full Search Vector Quantization.” IEEE TIP, 1994. https://doi.org/10.1109/83.287025
- Andre et al. “Derived Codebooks for High-Accuracy Nearest Neighbor Search.” 2019. https://arxiv.org/abs/1905.06900
- Douze et al. “Polysemous Codes.” ECCV 2016. https://arxiv.org/abs/1609.01882
- Chou, Lookabaugh, and Gray. “Optimal Pruning with Applications to Tree-Structured Source Coding and Modeling.” IEEE TIT, 1989. https://doi.org/10.1109/18.32124
- Andre et al. “Quicker ADC: Unlocking the Hidden Potential of Product Quantization with SIMD.” https://arxiv.org/abs/1812.09162
- Lee and Song. “Q-Palette: Fractional-Bit Quantizers Toward Optimal Bit Allocation for Efficient LLM Deployment.” 2025. https://arxiv.org/abs/2509.20214
- Lee and Kim. “FibQuant: Universal Vector Quantization for Random-Access KV-Cache Compression.” 2026. https://arxiv.org/abs/2605.11478
- Reviewed decision: saq-a4-prefix-novelty-gate@7f4c001.